

TECHNICAL PROJECT REPORT

Mini Automotive ECU Network with UDS Diagnostics

CAN communication, end-to-end protection and ISO-TP on an
STM32G4 Sensor ECU and an STM32MP157 Control ECU

By
Tu Dinh Huy

Interim report – milestones M1 and M2

Source code:

<https://github.com/huydinhischanging/mini-automotive-ecu-uds>

Ho Chi Minh City, Vietnam
September 2026

TABLE OF CONTENTS

LIST OF FIGURES	3
LIST OF TABLES	4
LIST OF ABBREVIATIONS	5
ABSTRACT	6
1 INTRODUCTION	7
1.1 Background	7
1.2 Problem Statement and Motivation	7
1.3 Objectives	7
1.4 Scope of this Report	8
1.5 Report Structure	8
2 TECHNICAL BACKGROUND	9
2.1 Controller Area Network	9
2.1.1 Bit timing	9
2.2 ISO-TP (ISO 15765-2)	10
2.3 Unified Diagnostic Services (ISO 14229)	10
2.4 End-to-End Protection	10
2.5 Layered ECU Software	11
2.6 STM32MP157 Heterogeneous Architecture	11
3 SYSTEM DESIGN	12
3.1 Requirements	12
3.2 System Architecture and Hardware	12
3.3 CAN Configuration	13
3.3.1 Bit timing	13
3.3.2 CAN matrix	14
3.4 Software Architecture	14
3.5 Sensor ECU Design	15
3.6 Control ECU Design	15
3.7 Communication Modules	16
3.7.1 E2E protection	16
3.7.2 ISO-TP	16
3.8 Verification Strategy	17
4 IMPLEMENTATION AND RESULTS	18
4.1 Development Environment	18
4.2 Unit Tests	18

4.3	Milestone M1: Sensor ECU in Loopback	19
4.4	Milestone M2: Control ECU in Loopback	20
4.5	Bring-up of the Control ECU	21
4.5.1	Issue 1: missing SysTick handler	21
4.5.2	Issue 2: FDCAN kernel clock stuck in a glitch-free multiplexer . . .	21
4.5.3	Other observations	22
4.6	Resource Usage	22
5	DISCUSSION AND EVALUATION	23
5.1	Evaluation against the Requirements	23
5.2	What Loopback Tests Do and Do Not Prove	23
5.3	Lessons Learned	23
5.4	Limitations	24
6	CONCLUSION AND FUTURE WORK	25
6.1	Conclusion	25
6.2	Future Work	25
	REFERENCES	27
	Appendices	28
A	Frame Layouts	28
B	Build, Deployment and Diagnosis Procedures	29
B.1	Host unit tests	29
B.2	Sensor ECU	29
B.3	Control ECU (Cortex-M4 on the DK1)	29
B.4	Reading M4 memory and registers from Linux	30

LIST OF FIGURES

2.1	Segmented ISO-TP transfer of a 20-byte message	10
3.1	Target network architecture. The dashed tester link is future work.	13
3.2	AUTOSAR-inspired software layers	14
3.3	Control ECU data flow	15
4.1	Sensor ECU log in FDCAN internal loopback (one line per second)	19
4.2	Control ECU log on the STM32MP157 Cortex-M4 in FDCAN internal loopback	20

LIST OF TABLES

1.1	Project status at the time of writing	8
3.1	System requirements	12
3.2	Hardware	13
3.3	Bit-timing configuration (500 kbit/s)	13
3.4	CAN matrix	14
3.5	Faults monitored by the Control ECU	16
4.1	Tools	18
4.2	Unit test results	19
4.3	Milestone M1 results	20
4.4	Milestone M2 results	20
4.5	FDCAN clock state left by Linux after boot	21
4.6	Memory footprint (Debug builds)	22
5.1	Requirement coverage	23
A.1	Frame 0x100 SENSOR_SPEED (DLC 8, 10 ms)	28
A.2	Frame 0x101 SENSOR_ALIVE (DLC 8, 100 ms)	28

LIST OF ABBREVIATIONS

ACK	Acknowledge (CAN bit acknowledging a correctly received frame)
ADC	Analog-to-Digital Converter
AUTOSAR	AUTomotive Open System ARchitecture
BS	Block Size (ISO-TP flow-control parameter)
CAN	Controller Area Network
CCU	Clock Calibration Unit (of FDCAN1 on the STM32MP1)
CF	Consecutive Frame (ISO-TP)
CRC	Cyclic Redundancy Check
DLC	Data Length Code
DTC	Diagnostic Trouble Code
E2E	End-to-End (communication protection)
ECU	Electronic Control Unit
FC	Flow Control frame (ISO-TP)
FDCAN	Flexible Data-rate CAN controller (ST implementation of Bosch M_CAN)
FF	First Frame (ISO-TP)
HAL	Hardware Abstraction Layer (STM32Cube)
HSE	High-Speed External oscillator
HSI	High-Speed Internal RC oscillator
ISO-TP	ISO 15765-2 transport protocol
ISR	Interrupt Service Routine
LSB	Least Significant Bit
MCAL	Microcontroller Abstraction Layer (AUTOSAR)
MCU	Microcontroller Unit
MPU	Microprocessor Unit
NBT	Nominal Bit Time, in time quanta
NRC	Negative Response Code (UDS)
PLL	Phase-Locked Loop
PWM	Pulse-Width Modulation
RTOS	Real-Time Operating System
SF	Single Frame (ISO-TP)
SJW	Synchronisation Jump Width
SP	Sample Point
STmin	Minimum Separation Time between consecutive frames (ISO-TP)
tq	Time quantum
UART	Universal Asynchronous Receiver-Transmitter
UDS	Unified Diagnostic Services (ISO 14229)

ABSTRACT

This report describes the design and the first two milestones of a scaled-down in-vehicle network built from low-cost development boards. A *Sensor ECU* on an STM32G431 (NUCLEO-G431RB) samples a potentiometer that emulates vehicle speed and publishes it on a 500 kbit/s classic CAN bus every 10 ms. A *Control ECU* runs FreeRTOS on the Cortex-M4 core of an STM32MP157D (STM32MP157D-DK1), next to Linux on the Cortex-A7 cores, and consumes the signal, monitors it for faults and drives a PWM output. A diagnostic tester using UDS (ISO 14229) over ISO-TP (ISO 15765-2) completes the target architecture.

The software follows an AUTOSAR-inspired layering in which only the MCAL layer touches the vendor HAL. Protocol modules (ISO-TP, end-to-end protection with an alive counter and CRC-8 SAE-J1850) are written in portable C and are tested on a PC with a small fault-injection-capable test harness: 34 unit tests with 173 checks pass, including a 4095-byte ISO-TP transfer between two protocol instances.

Both ECUs were verified on hardware in FDCAN internal loopback. The Sensor ECU sent more than 59 000 frames at the designed 110 frames/s with no dropped frame and no bus-off; the ADC reading was stable to ± 1 –2 LSB. The Control ECU processed 30 900 frames with zero CRC, sequence or queue-overflow errors while its output tracked the received speed. Bringing up the Control ECU uncovered two platform issues that were diagnosed by reading M4 memory and peripheral registers from Linux, without a hardware debugger: a SysTick handler missing from STM32CubeMX-generated code, and an FDCAN kernel clock stuck in a glitch-free multiplexer whose previous input had been disabled by Linux.

The remaining milestones are the physical two-node bus, the UDS server with DTC management, and a Python tester on Linux.

Keywords: Controller Area Network, ISO-TP, UDS, AUTOSAR E2E, STM32MP157, FreeRTOS, embedded C, unit testing.

Chapter 1

INTRODUCTION

1.1 Background

A modern vehicle contains dozens of electronic control units (ECUs) that exchange data over in-vehicle networks, most commonly the Controller Area Network (CAN) [1]. Each ECU samples sensors, runs control software and, just as importantly, supervises itself: it detects missing or implausible signals, stores diagnostic trouble codes (DTCs) and answers a workshop tester through a standardised diagnostic protocol, Unified Diagnostic Services (UDS) [2]. Because a CAN frame carries at most eight data bytes, diagnostic messages travel over the ISO-TP transport layer, which segments and reassembles longer messages [3].

These building blocks, deterministic communication, end-to-end protection of safety-relevant signals, fault management and diagnostics, are the daily material of automotive embedded software. They are rarely visible in hobby projects, which usually stop at reading a sensor and blinking a LED.

1.2 Problem Statement and Motivation

Professional automotive tool chains (AUTOSAR stacks, commercial CAN analysers, production ECUs) are expensive and hide most of the mechanisms behind configuration tools. A learner therefore has few opportunities to implement and debug the protocols end to end on real hardware. The goal of this project is to build a miniature but technically faithful ECU network from affordable development boards, implementing the protocols from the standards rather than from vendor black boxes, and to document the engineering decisions in a way that can be reviewed.

1.3 Objectives

- 01.** Build a two-ECU classic CAN network at 500kbit/s with a correct, justified bit-timing configuration on two different STM32 families.
- 02.** Protect the cyclic sensor signal end to end (alive counter and CRC) and detect timeout, out-of-range and corrupted data at the receiver, with a defined safe state.
- 03.** Implement ISO-TP and a UDS server (sessions, tester present, read data by identifier, read and clear DTCs, ECU reset, negative responses) on the Control ECU.

- O4.** Structure the firmware in AUTOSAR-like layers so that protocol code is hardware independent and unit tested on a PC.
- O5.** Run the Control ECU on the Cortex-M4 of a heterogeneous STM32MP157, alongside Linux, as in modern domain controllers.

1.4 Scope of this Report

This is an interim report. Table 1.1 summarises what is complete and verified at the time of writing. All results in Chapter 4 refer to the completed items only; the remaining items are described as design and future work.

Table 1.1: Project status at the time of writing

Work item	Evidence	Status
Sensor ECU firmware (ADC, cyclic CAN, E2E, fault injection)	M1 hardware test	✓
ISO-TP transport layer	26 unit tests	✓
E2E protection library (shared by both ECUs)	8 unit tests	✓
Control ECU firmware on the Cortex-M4 (FreeRTOS, E2E check, fault monitor, safe-state output)	M2 hardware test	✓
Two-node physical CAN bus with transceivers	–	planned (M3)
UDS server and DTC manager	–	planned
Python tester on Linux (SocketCAN)	–	planned
Static analysis (cppcheck/MISRA) and demo video	–	planned

1.5 Report Structure

Chapter 2 reviews the relevant protocols and the STM32MP1 heterogeneous architecture. Chapter 3 presents the hardware, the CAN configuration and the software design of both ECUs. Chapter 4 reports the implementation, the unit tests and the two hardware milestones, including the diagnosis of two bring-up issues. Chapter 5 evaluates the results against the objectives and discusses limitations, and Chapter 6 concludes with the remaining work. The appendices contain the frame layouts and the build and diagnosis procedures.

Chapter 2

TECHNICAL BACKGROUND

This chapter summarises the standards and platform features that the design relies on. It is not a complete tutorial; the cited specifications are normative.

2.1 Controller Area Network

CAN is a multi-master serial bus with non-destructive bitwise arbitration: when several nodes start transmitting, the frame with the lowest identifier wins and the others retry automatically [1], [4]. Every receiver acknowledges a correct frame in the ACK slot, so a transmitter that is alone on the bus sees an acknowledgement error and keeps retrying. Each node maintains transmit and receive error counters; above 255 transmit errors it enters the *bus-off* state and stops participating until it has observed 128 occurrences of 11 recessive bits.

2.1.1 Bit timing

A bit is divided into time quanta (tq) of length $t_q = \text{prescaler}/f_{\text{kernel}}$. The nominal bit time is

$$\text{NBT} = 1 + \text{Seg1} + \text{Seg2} \quad [\text{tq}], \quad \text{bitrate} = \frac{f_{\text{kernel}}}{\text{prescaler} \cdot \text{NBT}}, \quad (2.1)$$

where the leading quantum is the synchronisation segment, Seg1 combines propagation and phase segment 1, and Seg2 is phase segment 2. The bus is sampled at the sample point

$$\text{SP} = \frac{1 + \text{Seg1}}{\text{NBT}}, \quad (2.2)$$

for which automotive networks typically use about 87.5%. The synchronisation jump width (SJW) limits how far a node may re-align its bit phase on each recessive-to-dominant edge. Because every node samples with its own oscillator, the tolerated clock deviation is bounded by [1]

$$\Delta f \leq \min \left(\frac{\text{SJW}}{20 \text{ NBT}}, \frac{\min(\text{Seg1}, \text{Seg2})}{2 (13 \text{ NBT} - \text{Seg2})} \right). \quad (2.3)$$

For the configurations in this project the bound is below 0.5%, which rules out on-chip RC oscillators (typically $\pm 1\%$) and motivates crystal clocks on both ECUs.

2.2 ISO-TP (ISO 15765-2)

ISO-TP transports messages of up to 4095 bytes over classic CAN frames [3]. The first byte (protocol control information) identifies four frame types:

- **Single Frame (SF)** 0L: up to 7 data bytes.
- **First Frame (FF)** 1L LL: 12-bit total length and 6 data bytes.
- **Consecutive Frame (CF)** 2N: 7 data bytes with a 4-bit sequence number that wraps from 15 to 0.
- **Flow Control (FC)** 3S BS STmin: sent by the receiver with a flow status (continue, wait, overflow), a block size BS (number of CFs before the next FC, 0 = unlimited) and the minimum separation time STmin between CFs.

Timers supervise the exchange: the sender waits at most N_Bs for a flow control, the receiver at most N_Cr for the next consecutive frame. Figure 2.1 shows a segmented transfer.

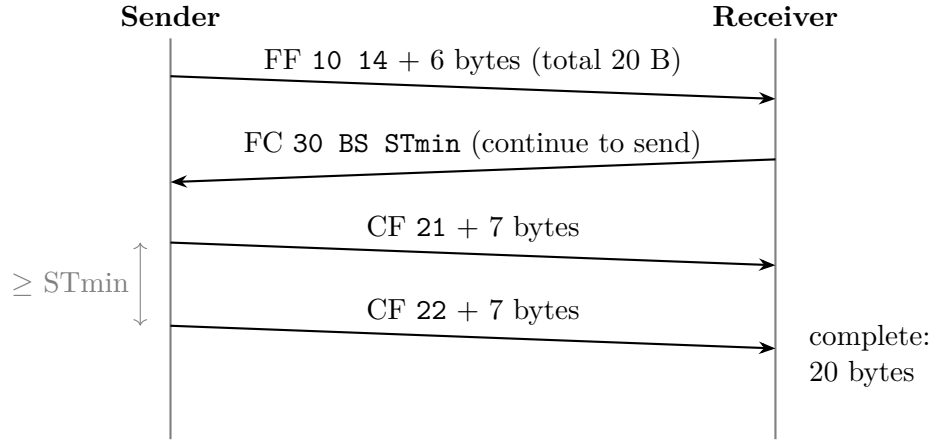


Figure 2.1: Segmented ISO-TP transfer of a 20-byte message

2.3 Unified Diagnostic Services (ISO 14229)

UDS defines a request/response protocol between a tester and an ECU [2]. A request starts with a service identifier (SID); a positive response echoes $SID + 0x40$, and a negative response is **7F SID NRC** with a negative response code such as 0x11 (service not supported) or 0x31 (request out of range). The services targeted by this project are Diagnostic Session Control (0x10), ECU Reset (0x11), Clear Diagnostic Information (0x14), Read DTC Information (0x19), Read Data By Identifier (0x22) and Tester Present (0x3E). On vehicles the physical request and response identifiers of the engine ECU are commonly 0x7E0 and 0x7E8; the project reuses this pair.

2.4 End-to-End Protection

Bus-level CAN checks detect transmission errors on the wire but not faults inside an ECU, such as a stuck task that keeps resending an old buffer or corrupted RAM. AUTOSAR E2E profiles therefore add, inside the payload, an alive (sequence) counter and a CRC computed by the sender and verified by the receiver [5]. Profile 1 uses a 4-bit counter and CRC-8 SAE-J1850 (polynomial 0x1D) [6]. The receiver classifies each frame as correct,

repeated, with some frames lost, or with a wrong sequence, and discards data that fails the check.

2.5 Layered ECU Software

The AUTOSAR classic platform separates application software from basic software, and at the bottom of the basic software the microcontroller abstraction layer (MCAL) encapsulates all register access [7]. This project applies the same idea at a small scale: only MCAL drivers call the STM32 HAL; communication and service modules are portable C.

2.6 STM32MP157 Heterogeneous Architecture

The STM32MP157 combines two Cortex-A7 cores running Linux with a Cortex-M4 microcontroller core [8]. In the *production* boot mode used here, Linux starts first, owns the clock tree and loads the M4 firmware from the file system through the **remoteproc** framework. The M4 has no flash of its own; its code runs from internal SRAM and must be reloaded after each power cycle. Peripherals are assigned to one of the cores; a peripheral assigned to the M4 is disabled in the Linux device tree, but its kernel clock is still generated by the reset and clock controller (RCC), whose settings are left by Linux. This division of responsibilities is central to the bring-up issues analysed in Section 4.5.

Chapter 3

SYSTEM DESIGN

3.1 Requirements

Table 3.1 lists the requirements derived from the objectives in Chapter 1. They are referenced in the evaluation in Chapter 5.

Table 3.1: System requirements

ID	Requirement
R1	Classic CAN 2.0A, 11-bit identifiers, 500 kbit/s, sample point close to 87.5 % on every node.
R2	The Sensor ECU publishes the speed signal every 10 ms and a node heartbeat every 100 ms.
R3	Cyclic frames carry an alive counter and a CRC; the receiver rejects corrupted, repeated and out-of-sequence data.
R4	The Control ECU detects a missing speed signal within 100 ms, an out-of-range speed and an E2E error, and then drives its output to a safe state (0 %).
R5	Fault injection on both ECUs by push button, without changing wiring.
R6	Diagnostic access through UDS over ISO-TP on 0x7E0/0x7E8, with negative responses.
R7	Only the MCAL layer accesses the HAL; protocol modules are hardware independent and unit tested on a PC.
R8	Bus-off is recovered automatically after a delay, without resetting the ECU.

3.2 System Architecture and Hardware

Figure 3.1 shows the target network. Table 3.2 lists the hardware.

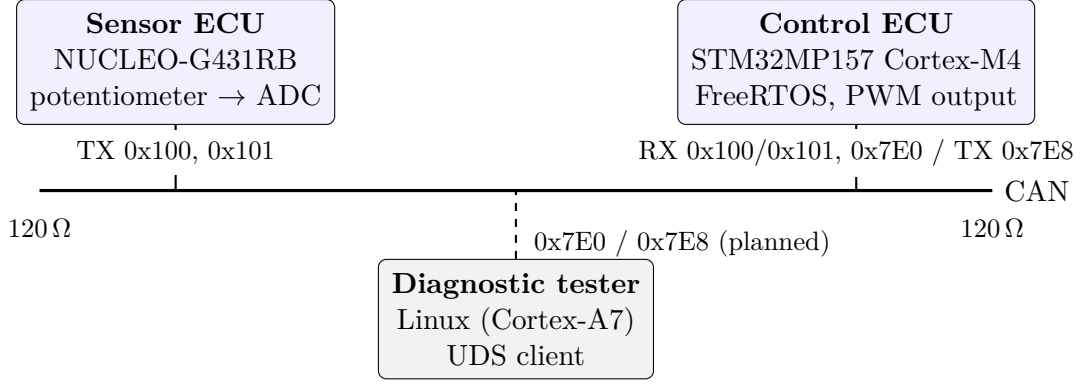


Figure 3.1: Target network architecture. The dashed tester link is future work.

Table 3.2: Hardware

Item	Use
NUCLEO-G431RB (STM32G431RB, Cortex-M4F, 170 MHz) [9], [10]	Sensor ECU; on-board ST-LINK for flashing and a virtual COM port for logs
STM32MP157D-DK1 (2×Cortex-A7 + Cortex-M4) [8], [11]	Control ECU on the M4, Linux (OpenSTLinux, kernel 6.6) on the A7 cores
SN65HVD230 3.3 V CAN transceiver (2×) [12]	Physical layer; 3.3 V parts avoid level shifting on 3.3 V MCU pins
10 kΩ potentiometer, push buttons, LED + 220 Ω	Speed input, fault injection, PWM output
USB-to-UART adapter	Log of the M4 firmware (UART7)

3.3 CAN Configuration

3.3.1 Bit timing

The two controllers run from different kernel clocks, so each node has its own timing. Both are derived from a crystal (HSE) because the tolerance bound of Equation 2.3 is below 0.5 % for both configurations. Table 3.3 gives the values.

Table 3.3: Bit-timing configuration (500 kbit/s)

	Sensor ECU (STM32G431)	Control ECU (STM32MP157)
Kernel clock	170 MHz (PCLK1, from 24 MHz HSE)	24 MHz (HSE)
Prescaler	10	3
t_q	58.8 ns	125 ns
Seg1 / Seg2 / SJW	29 / 4 / 4	13 / 2 / 2
NBT	34 t_q	16 t_q
Bit rate	500 000 bit/s	500 000 bit/s
Sample point	88.2 %	87.5 %
Tolerance bound (Eq. 2.3)	0.46 %	0.49 %

On the STM32G431, 170 MHz cannot be divided into a 16-quantum bit at 500 kbit/s, so 34 t_q were used; the resulting 88.2 % sample point is within a fraction of a quantum

of 87.5 %. On the STM32MP157 the 24 MHz HSE gives exactly 16 tq and 87.5 %. The kernel clock that Linux selects by default (PLL4_R, 74.25 MHz) cannot produce 500 kbit/s exactly ($74.25 \text{ MHz} / 500 \text{ kHz} = 148.5$); the Control ECU therefore switches the FDCAN kernel clock to HSE at start-up (Section 4.5) and keeps a fallback timing for 74.25 MHz with a 0.34 % error.

3.3.2 CAN matrix

Table 3.4 lists the identifiers. The byte layout is defined once in `common/can/can_matrix.h`, which both firmware projects compile, so the layouts cannot drift apart. The detailed layout is in Appendix A.

Table 3.4: CAN matrix

ID	Name	Sender	Cycle	Content
0x100	SENSOR_SPEED	Sensor ECU	10 ms	speed (0.1 km/h), raw ADC, status, alive counter, CRC-8
0x101	SENSOR_ALIVE	Sensor ECU	100 ms	node state, uptime, bus-off count, alive counter, CRC-8
0x7E0	UDS request	tester	event	ISO-TP, physical addressing to the Control ECU
0x7E8	UDS response	Control ECU	event	ISO-TP response

Multi-byte signals are big-endian (Motorola byte order), as is usual in automotive signal databases, and the speed is an unsigned integer in 0.1 km/h so that no floating-point value travels on the bus.

3.4 Software Architecture

Figure 3.2 shows the layering. MCAL drivers wrap the STM32 HAL; everything above them is portable C. The shared modules in `common/` are linked into both STM32CubeIDE projects as a linked folder, so the firmware and the PC unit tests compile the very same source files.

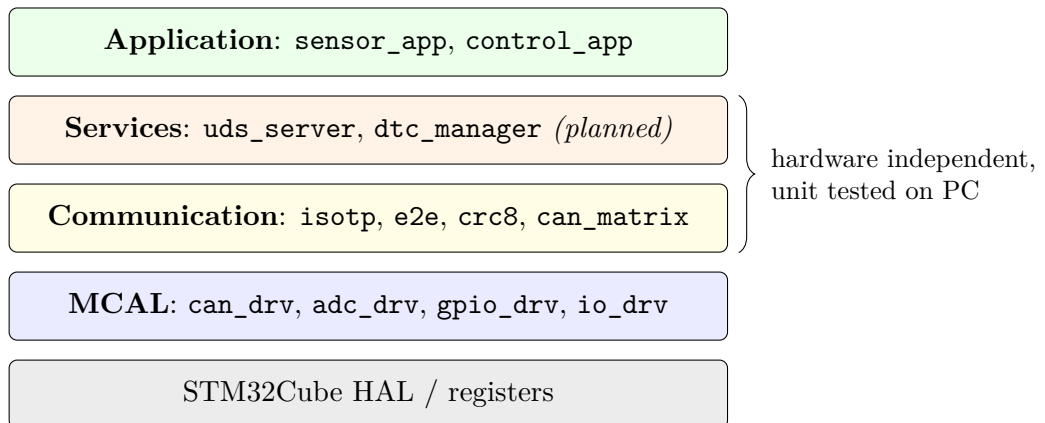


Figure 3.2: AUTOSAR-inspired software layers

3.5 Sensor ECU Design

The Sensor ECU runs bare-metal. TIM6 raises a flag every 10 ms; the main loop consumes it and runs one task cycle. The interrupt routine only sets the flag, so ADC conversions, CAN writes and logging never execute in interrupt context, and a missed deadline is counted as an overrun. Each cycle:

1. debounces push button B1 (30 ms) and advances the fault-injection mode on each press;
2. samples the potentiometer (12-bit ADC, 92.5-cycle sampling time) and filters it with an 8-sample moving average;
3. scales the value to 0–250.0 km/h and sends frame 0x100 protected by the E2E module;
4. every tenth cycle sends frame 0x101;
5. runs the bus-off recovery state machine and the status LED.

Sampling time. The potentiometer wiper has a source impedance of up to 2.5 k Ω (two 5 k Ω halves in parallel). With a sample capacitor of about 5 pF, reaching 12-bit accuracy needs roughly ten time constants, about 120 ns, which the default 2.5-cycle sampling time at 42.5 MHz (59 ns) does not provide. 92.5 cycles (2.2 μ s) gives a large margin at negligible cost, since only one sample is taken every 10 ms.

Fault injection. Button B1 cycles through three modes: **NONE**; **SILENT**, which stops frame 0x100 so that the receiver’s timeout detection is exercised; and **OUT_OF_RANGE**, which sends 300.0 km/h. The status byte of 0x100 flags active fault injection so that injected faults can be told apart from real ones in a bus trace.

Bus-off recovery. On bus-off the FDCAN hardware sets **CCCR.INIT**. The driver waits 100 ms and then clears **INIT**, which starts the standard recovery sequence of 128 $\times$ 11 recessive bits. The delay prevents a faulty node from flooding the bus with error frames.

3.6 Control ECU Design

The Control ECU runs FreeRTOS [13] on the Cortex-M4. Figure 3.3 shows the data flow.

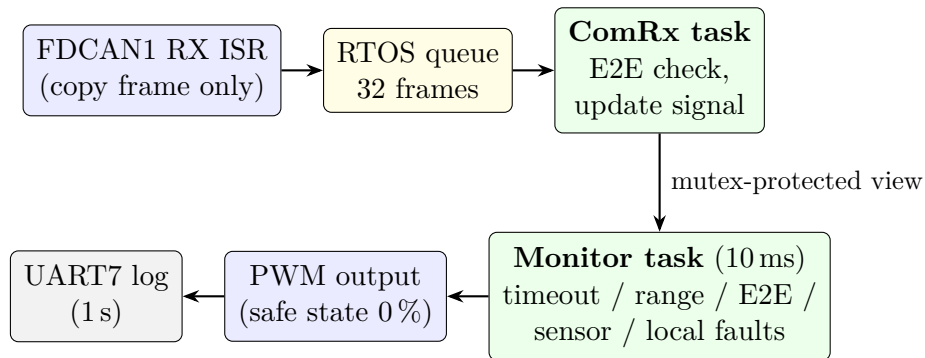


Figure 3.3: Control ECU data flow

- **Interrupt level.** The FDCAN receive callback copies each frame into an RTOS queue and returns; a full queue is counted rather than blocking.

- **ComRx task** (above-normal priority) runs the E2E check on 0x100, updates a shared view of the sensor signal under a mutex and counts E2E outcomes. Frames 0x7E0/0x7DF are routed to the future UDS server.
- **Monitor task** (normal priority, 10 ms period with `osDelayUntil`) evaluates five faults, applies the safe state, runs bus-off recovery and prints a status line every second.

Table 3.5 lists the monitored faults. When a fault that invalidates the speed signal is active, the output is forced to 0 % instead of using stale or implausible data. The E2E fault heals only after ten consecutive valid frames to avoid chattering.

Table 3.5: Faults monitored by the Control ECU

Fault	Condition	Safe state
SPEED_TIMEOUT	no valid 0x100 for more than 100 ms (500 ms grace after start)	yes
SPEED_RANGE	received speed above 250.0 km/h	yes
SPEED_E2E	CRC or sequence error; healed after 10 valid frames	yes
SENSOR_ADC	Sensor ECU reports an ADC read error in its status byte	no
OUTPUT_LOCAL	toggled by button USER1 (simulated actuator fault)	yes

In a later milestone each fault becomes a DTC with an ISO 14229 status byte in the DTC manager, readable and clearable over UDS.

3.7 Communication Modules

3.7.1 E2E protection

`E2e_Protect()` writes the 4-bit alive counter to byte 6 and the CRC-8 SAE-J1850 of bytes 0–6 to byte 7. `E2e_Check()` verifies the CRC first, so a corrupted frame never modifies the receiver state, and then classifies the counter difference modulo 16: 0 is *repeated*, 1 is *OK*, 2–3 is *OK with some frames lost*, and larger jumps are rejected as *wrong sequence* with resynchronisation on the next frame.

3.7.2 ISO-TP

The ISO-TP module keeps independent sender and receiver state machines per link. It is hardware independent: the CAN send function and a millisecond clock are injected through a configuration structure, which makes the timing behaviour fully controllable in unit tests. Notable design points:

- all frames are padded to 8 bytes with 0xCC;
- STmin is enforced with one extra tick of margin, because with a 1 ms clock a frame sent at $x.99$ ms and the next at $(x + 1).00$ ms would be only 0.01 ms apart;
- STmin values in the 100–900 μ s range are rounded up to 1 ms, and reserved values are treated as the 127 ms maximum, as required by the standard;
- if the CAN driver is busy (three TX FIFO slots on the STM32), a frame is retried on the next cycle instead of being dropped;
- N_Bs and N_Cr timeouts, a limit on consecutive FC.WAIT frames and FC.OVFLW handling abort a transfer with an explicit result code.

3.8 Verification Strategy

Verification proceeds bottom-up:

1. **Unit tests on a PC** for all hardware-independent modules, using a fake CAN driver and a fake clock. Compiled with `-Wall -Wextra -Wpedantic -Wshadow -Wconversion`.
2. **FDCAN internal loopback** on each ECU, which exercises bit-timing configuration, filters, interrupts and the full software path without a transceiver.
3. **Two-node bus** with transceivers, checked with a logic analyser (next milestone).
4. **System test** with the UDS tester (final milestone).

Each hardware test records the UART log as evidence, together with a pass criterion defined before the run.

Chapter 4

IMPLEMENTATION AND RESULTS

4.1 Development Environment

Table 4.1: Tools

Tool	Use
STM32CubeMX 6.18	peripheral, clock and middleware configuration of both projects
STM32CubeIDE 2.2 / GNU Arm toolchain 14.3	firmware build
STM32CubeProgrammer CLI	flashing the NUCLEO-G431RB over SWD
OpenSTLinux (kernel 6.6), <code>remoteproc</code> , SSH over USB	loading and starting the M4 firmware on the DK1
MSYS2 GCC and GNU Make	host unit tests
Git / GitHub	version control; each verified milestone is tagged

The M4 firmware is deployed by copying the ELF file to `/lib/firmware` and writing to the `remoteproc` sysfs interface (Appendix B).

4.2 Unit Tests

The hardware-independent modules are tested on a PC with a minimal self-written test harness (no external framework), a fake CAN driver that records every transmitted frame and can simulate a busy driver, and a fake millisecond clock that only advances when a test says so. Table 4.2 summarises the suites.

Table 4.2: Unit test results

Suite	Tests	Checks	Scope
ISO-TP	26	115	SF/FF/CF/FC in both directions, padding, block size, STmin in ms and μ s encoding, N_Bs and N_Cr timeouts, FC.WAIT limit, FC.OVFLW, receive buffer overflow, wrong sequence number, SF interrupting a segmented reception, sequence number wrap-around (200 bytes), driver-busy retry, and a 4095-byte end-to-end transfer between two links on a simulated bus
E2E	8	58	CRC-8 catalogue value ("123456789" $\rightarrow$ 0x4B), counter wrap 15 $\rightarrow$ 0, single-bit corruption, repeated frame, lost frames and resynchronisation, short frame
Total	34	173	all pass, 0 failures

The 4095-byte end-to-end test also checks the exact frame count: one first frame, 585 consecutive frames and 74 flow-control frames for a block size of 8. An error in the expected count found while writing this test (584 instead of 585 consecutive frames, because $4089 = 584 \times 7 + 1$) illustrates why such arithmetic belongs in automated tests rather than in comments.

4.3 Milestone M1: Sensor ECU in Loopback

The Sensor ECU firmware was flashed with the STM32CubeProgrammer CLI and run in FDCAN internal loopback, where the controller receives its own frames. The pass criterion was: transmit rate as designed, every 0x100 frame received back, no dropped frame, no bus-off, and a stable ADC reading that follows the potentiometer. Figure 4.1 shows the log and Table 4.3 the results.

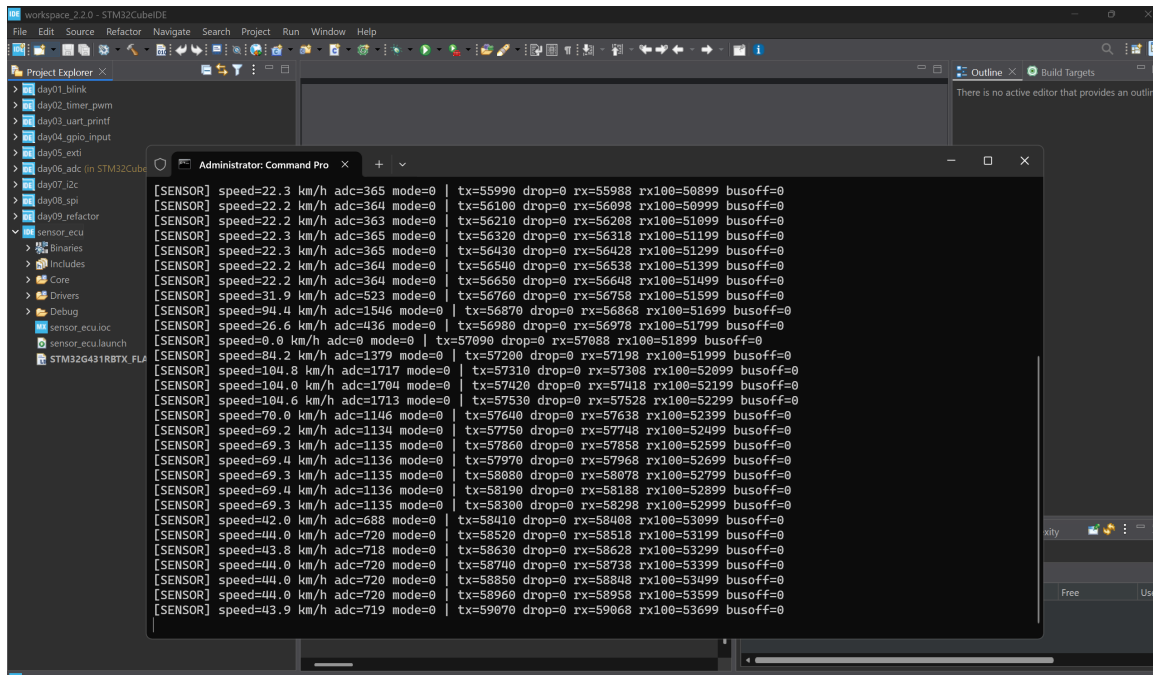


Figure 4.1: Sensor ECU log in FDCAN internal loopback (one line per second)

Table 4.3: Milestone M1 results

Check	Observation	Result
Transmit rate	+110 frames per second ($100 \times 0x100 + 10 \times 0x101$)	✓
Loopback reception	$rx100 + 100$ per second; $rx = tx - 2$ constant (two frames in flight when the log is printed)	✓
Endurance	more than 59 000 frames, $drop=0$, $busoff=0$	✓
ADC	follows the potentiometer (0 to 104.8 km/h observed during the sweep); $\pm 1-2$ LSB at rest	✓
Scaling	e.g. $ADC\ 865 \rightarrow 865 \times 2500/4095 = 528 \rightarrow 52.8\text{ km/h}$	✓
Fault injection (B1)	not yet exercised on hardware	open

4.4 Milestone M2: Control ECU in Loopback

In loopback the Control ECU also plays the Sensor ECU: its monitor task sends an E2E-protected $0x100$ frame every 10 ms with a speed ramp from 0 to 250 km/h, so the whole chain from the FDCAN interrupt to the PWM output is exercised on the DK1 alone. Figure 4.2 shows the log and Table 4.4 the results.

```

off=0 qovf=0 uds=0 heap=11032
[CTRL] speed=43.6 km/h age=10ms pwm=17.4% | e2e ok=29599 lost=0 crc=0 rep=0 seq=0 | faults=0x00 | rx=29599 tx=29600 buso
ff=0 qovf=0 uds=0 heap=11032
[CTRL] speed=93.6 km/h age=10ms pwm=37.4% | e2e ok=29699 lost=0 crc=0 rep=0 seq=0 | faults=0x00 | rx=29699 tx=29700 buso
ff=0 qovf=0 uds=0 heap=11032
[CTRL] speed=143.6 km/h age=10ms pwm=57.4% | e2e ok=29799 lost=0 crc=0 rep=0 seq=0 | faults=0x00 | rx=29799 tx=29800 bus
ff=0 qovf=0 uds=0 heap=11032
[CTRL] speed=193.6 km/h age=10ms pwm=77.4% | e2e ok=29899 lost=0 crc=0 rep=0 seq=0 | faults=0x00 | rx=29899 tx=29900 bus
ff=0 qovf=0 uds=0 heap=11032
[CTRL] speed=243.6 km/h age=10ms pwm=97.4% | e2e ok=29999 lost=0 crc=0 rep=0 seq=0 | faults=0x00 | rx=29999 tx=30000 bus
ff=0 qovf=0 uds=0 heap=11032
[CTRL] speed=43.5 km/h age=10ms pwm=17.4% | e2e ok=30099 lost=0 crc=0 rep=0 seq=0 | faults=0x00 | rx=30099 tx=30100 buso
ff=0 qovf=0 uds=0 heap=11032
[CTRL] speed=93.5 km/h age=10ms pwm=37.4% | e2e ok=30199 lost=0 crc=0 rep=0 seq=0 | faults=0x00 | rx=30199 tx=30200 buso
ff=0 qovf=0 uds=0 heap=11032
[CTRL] speed=143.5 km/h age=10ms pwm=57.4% | e2e ok=30299 lost=0 crc=0 rep=0 seq=0 | faults=0x00 | rx=30299 tx=30300 bus
ff=0 qovf=0 uds=0 heap=11032
[CTRL] speed=193.5 km/h age=10ms pwm=77.4% | e2e ok=30399 lost=0 crc=0 rep=0 seq=0 | faults=0x00 | rx=30399 tx=30400 bus
ff=0 qovf=0 uds=0 heap=11032
[CTRL] speed=243.5 km/h age=10ms pwm=97.4% | e2e ok=30499 lost=0 crc=0 rep=0 seq=0 | faults=0x00 | rx=30499 tx=30500 bus
ff=0 qovf=0 uds=0 heap=11032
[CTRL] speed=43.4 km/h age=10ms pwm=17.3% | e2e ok=30599 lost=0 crc=0 rep=0 seq=0 | faults=0x00 | rx=30599 tx=30600 buso
ff=0 qovf=0 uds=0 heap=11032
[CTRL] speed=93.4 km/h age=10ms pwm=37.3% | e2e ok=30699 lost=0 crc=0 rep=0 seq=0 | faults=0x00 | rx=30699 tx=30700 buso
ff=0 qovf=0 uds=0 heap=11032
[CTRL] speed=143.4 km/h age=10ms pwm=57.3% | e2e ok=30799 lost=0 crc=0 rep=0 seq=0 | faults=0x00 | rx=30799 tx=30800 bus
ff=0 qovf=0 uds=0 heap=11032
[CTRL] speed=193.4 km/h age=10ms pwm=77.3% | e2e ok=30899 lost=0 crc=0 rep=0 seq=0 | faults=0x00 | rx=30899 tx=30900 bus
ff=0 qovf=0 uds=0 heap=11032

```

Figure 4.2: Control ECU log on the STM32MP157 Cortex-M4 in FDCAN internal loopback

Table 4.4: Milestone M2 results

Check	Observation	Result
Reception	e2e ok +100 per second, $rx = tx - 1$	✓
E2E	30 900 frames: 0 CRC, 0 lost, 0 repeated, 0 sequence errors	✓
Signal freshness	age = 10 ms (timeout threshold 100 ms)	✓
Output	PWM tracks speed, e.g. $193.4\text{ km/h} \rightarrow 77.3\%$	✓
Faults	timeout active at start-up, cleared automatically once data arrives; no active fault afterwards	✓
RTOS	0 queue overflows; 11 032 bytes of the 16 KB heap free	✓

4.5 Bring-up of the Control ECU

The first run of the Control ECU on the DK1 did nothing observable. No debugger connection to the M4 was available in production mode (the A7 owns the M4 reset), and the UART log was not yet readable. The firmware state was therefore inspected from Linux, which can read the M4 SRAM (aliased at 0x30020000) and all peripheral registers through `/dev/mem`; symbol addresses and structure offsets were taken from the ELF file with `nm` and `gdb`. Two independent defects were found.

4.5.1 Issue 1: missing SysTick handler

Observation. `xSchedulerRunning` = 1 and the current task was the idle task, but the FreeRTOS tick count `xTickCount` stayed at 0 and the HAL tick `uwTick` was frozen at 19. All application tasks were blocked waiting for a tick that never came.

Root cause. STM32CubeMX generated `USE_CUSTOM_SYSTICK_HANDLER_IMPLEMENTATION` = 1, which tells the CMSIS-RTOS2 layer not to define `SysTick_Handler` because the interrupt file will. With TIM7 selected as HAL time base, however, the interrupt file contained no such handler. The symbol table showed `SysTick_Handler` as a weak alias of `Default_Handler`, an endless loop: the first RTOS tick trapped the CPU at the SysTick priority, which also blocked the TIM7 interrupt and froze `uwTick`.

Fix. The flag is overridden to 0 inside a `USER CODE` section of `FreeRTOSConfig.h`, which STM32CubeMX preserves on regeneration, so that `cmsis_os2.c` provides the standard handler. After the fix the symbol is a strong definition and the tick advances at 1 kHz.

4.5.2 Issue 2: FDCAN kernel clock stuck in a glitch-free multiplexer

Observation. After the first fix the tasks ran, but `CCCR.INIT` remained 1, no frame was transmitted and the HAL state was `BUSY` with no error code, meaning that `HAL_FDCAN_Start()` had succeeded. Clearing `INIT` directly from Linux had no effect either, which pointed to a missing protocol clock rather than to the firmware.

Investigation. Register reads after a fresh Linux boot showed the state left by Linux (Table 4.5). The clock calibration unit of FDCAN1 was in bypass (`CCCFG.BCC` = 1) and therefore not involved.

Table 4.5: FDCAN clock state left by Linux after boot

Register field	Value	Meaning
<code>RCC_FDCANCKSEL</code>	3	FDCAN kernel clock taken from <code>PLL4_R</code>
<code>RCC_PLL4CR.DIVREN</code>	0	<code>PLL4_R</code> output disabled (unused by Linux)
<code>RCC_OCENSETR.HSEKERON</code>	0	HSE kernel clock output not enabled
<code>RCC_TZCR.TZEN</code>	1	RCC in secure mode; oscillator <i>disable</i> register is secure-only

Root cause. STM32 kernel clock multiplexers are glitch-free: a switch completes only while both the current and the new input are running [8]. The firmware switched the multiplexer from `PLL4_R` to HSE while `PLL4_R` was disabled, so the switch never

completed and the FDCAN core was left without any clock. The hypothesis was confirmed experimentally: enabling PLL4_R for a few milliseconds from Linux immediately let INIT clear and frames flow; PLL4_R was then disabled again and FDCAN kept running from HSE.

Fix. `CanDrv_PrepareKernelClock()`, called before the HAL initialises FDCAN, sets the HSE kernel clock enable, temporarily enables the output of the currently selected PLL if it is off, switches to HSE, waits 1 ms, restores the PLL output exactly as Linux left it, and reads back the effective kernel frequency. The fix was verified after a fresh reboot with no manual register access: the firmware alone brought FDCAN up at 24 MHz.

4.5.3 Other observations

- **FreeRTOS heap.** The default 3 KB heap was too small for five tasks (three application tasks plus the idle and timer-service tasks) with newlib re-entrancy enabled; it was raised to 16 KB, of which 11 KB remain free.
- **DLC encoding.** The STM32MP1 HAL encodes the data length in bits 19:16 of the header word, whereas the STM32G4 HAL uses the plain byte count. A compile-time assertion in each driver guards against silently using the wrong encoding.
- **HAL tick rate.** In production mode the HAL tick driven by TIM7 runs at about 2 kHz, because the timer clock configured by Linux differs from the value assumed by STM32CubeMX. The application uses the FreeRTOS tick, which is correct; the HAL tick is an open point.

4.6 Resource Usage

Table 4.6: Memory footprint (Debug builds)

Firmware	Code	RAM (data+bss)	Available
Sensor ECU (STM32G431)	31.6 KB	2.5 KB	128 KB flash / 32 KB RAM
Control ECU (M4, -00)	67.5 KB	22.9 KB	128 KB code SRAM / 128 KB data SRAM

Chapter 5

DISCUSSION AND EVALUATION

5.1 Evaluation against the Requirements

Table 5.1 maps the results of Chapter 4 to the requirements of Table 3.1. A requirement is marked as met only if it was demonstrated on hardware or by an automated test.

Table 5.1: Requirement coverage

ID	Evidence	Status
R1	Bit timing configured and confirmed in registers on both ECUs; exchange between the two nodes not yet tested	partly met
R2	M1: 100 + 10 frames per second	met
R3	E2E unit tests; M2: 30 900 frames checked	met
R4	Timeout set at start-up and cleared by data (M2); range and E2E detection implemented, not yet injected on hardware	partly met
R5	Implemented on both ECUs; not yet exercised on hardware	open
R6	ISO-TP complete and tested; UDS server not implemented	open
R7	Only MCAL files include the HAL; 34 PC unit tests on the shared modules	met
R8	Implemented on both ECUs; requires a physical bus to provoke bus-off	open

5.2 What Loopback Tests Do and Do Not Prove

Internal loopback validates a large part of the design: the controller configuration, the acceptance filters, the interrupt path, the frame packing, the E2E protection and the application logic. It does not validate the physical layer. In internal loopback the controller acknowledges its own frames, so an error in the transceiver wiring, the termination, or a mismatch in bit timing between the two nodes would stay invisible. The two ECUs deliberately use different kernel clocks and different numbers of time quanta; whether they interoperate is exactly what milestone M3 must show, preferably with a logic analyser measuring the bit time and the sample point on the bus.

5.3 Lessons Learned

Generated code is part of the system under test. The SysTick defect came from STM32CubeMX output, not from application code. It was only found because the investigation did not stop at “the tasks do not run” but checked the symbol table: a

weak symbol resolved to `Default_Handler` is a concrete, checkable fact. Putting the fix in a user section of the generated file keeps it across regeneration.

On a heterogeneous SoC the M4 inherits the clock tree of Linux. Code that works on a standalone MCU can fail on the STM32MP1 because peripherals assigned to the M4 still depend on clock settings chosen by Linux. The FDCAN defect was a consequence of a general hardware rule (glitch-free multiplexers need both inputs running) meeting a specific software state (an unused PLL output switched off). The clean long-term solution is to let Linux configure the FDCAN kernel clock through the device tree; the firmware-side switch with read-back is a pragmatic and verified workaround.

Debugging without a debugger. Reading M4 memory and peripheral registers from Linux through `/dev/mem`, guided by symbol addresses from the ELF file, proved to be an effective technique on this platform, and it also allowed hypotheses to be tested by writing registers (for example enabling `PLL4_R` to confirm the multiplexer theory) before changing any firmware.

Evidence before conclusions. Several intermediate hypotheses during bring-up were wrong: USB cables and adapters were suspected while simply re-seating the connectors resolved the connection problems, the clock calibration unit was suspected but found in bypass, and the role of the HSE kernel-clock gate could not be isolated because its disable register is secure-only. Recording which observations support each conclusion (Section 4.5) keeps the report honest about what was actually proven.

5.4 Limitations

- No two-node bus test yet; all hardware results are from internal loopback.
- UDS and DTC management are designed but not implemented; the Control ECU only counts diagnostic requests.
- The HAL tick of the M4 runs at about twice the intended rate in production mode.
- The FDCAN clock is reconfigured by the M4 rather than by the Linux device tree.
- Static analysis (cppcheck with MISRA rules) has not been run yet, and the firmware has not been measured for worst-case execution time or interrupt latency.

Chapter 6

CONCLUSION AND FUTURE WORK

6.1 Conclusion

This report presented the design of a miniature automotive ECU network and the completion of its first two milestones. A Sensor ECU on an STM32G431 publishes an end-to-end protected speed signal at 500 kbit/s with the designed timing, and a Control ECU on the Cortex-M4 of an STM32MP157 receives it under FreeRTOS, checks it, supervises it for faults and drives a safe-state output. The protocol layer is portable and backed by 34 passing unit tests, including a full-length ISO-TP transfer. Both ECUs were verified on hardware in internal loopback with zero communication errors over tens of thousands of frames.

The most instructive part of the work was the bring-up of the Control ECU on a heterogeneous SoC, where two defects outside the application code, a missing interrupt handler in generated code and a clock multiplexer blocked by the clock state left by Linux, were located by reading the running firmware's memory and registers from Linux and were fixed in a way that survives code regeneration and a cold boot.

6.2 Future Work

1. **M3 – physical bus.** Connect both ECUs through SN65HVD230 transceivers with $120\ \Omega$ termination at both ends, verify the $60\ \Omega$ bus resistance, and measure bit time and sample point with a logic analyser. Exercise fault injection, timeout and bus-off recovery on the real bus.
2. **UDS server.** Implement services 0x10, 0x11, 0x14, 0x19, 0x22 and 0x3E with negative responses and session timing on top of the existing ISO-TP module, with unit tests on the PC.
3. **DTC manager.** Store faults as DTCs with ISO 14229 status bits, confirmation and healing counters, readable and clearable over UDS.
4. **Tester.** A Python UDS client on Linux using SocketCAN, either through a USB-CAN adapter or the second FDCAN controller of the STM32MP157.
5. **Quality.** Static analysis with cppcheck and MISRA rules, a fix for the HAL tick rate, and moving the FDCAN clock configuration into the Linux device tree.

6. **Documentation.** A demonstration video of the complete fault-to-DTC-to-tester flow.

REFERENCES

- [1] *ISO 11898-1:2015 – Road vehicles – Controller area network (CAN) – Part 1: Data link layer and physical signalling*, Geneva: International Organization for Standardization, 2015.
- [2] *ISO 14229-1:2020 – Road vehicles – Unified diagnostic services (UDS) – Part 1: Application layer*, Geneva: International Organization for Standardization, 2020.
- [3] *ISO 15765-2:2016 – Road vehicles – Diagnostic communication over Controller Area Network (DoCAN) – Part 2: Transport protocol and network layer services*, Geneva: International Organization for Standardization, 2016.
- [4] *CAN Specification Version 2.0*, Robert Bosch GmbH, Stuttgart, 1991.
- [5] *Specification of SW-C End-to-End Communication Protection Library*, AUTOSAR Classic Platform, document AUTOSAR_SWS_E2ELibrary, AUTOSAR. [Online]. Available: <https://www.autosar.org/>
- [6] *SAE J1850 – Class B Data Communications Network Interface*, Source of the CRC-8 polynomial 0x1D used by AUTOSAR E2E Profile 1, SAE International.
- [7] *Layered Software Architecture*, AUTOSAR Classic Platform, document AUTOSAR_EXP_LayeredSoftwareArchitecture, AUTOSAR. [Online]. Available: <https://www.autosar.org/>
- [8] *RM0436: STM32MP157 advanced Arm-based 32-bit MPUs – Reference manual*, STMicroelectronics. [Online]. Available: <https://www.st.com/>
- [9] *UM2505: STM32G4 Nucleo-64 boards (MB1367) – User manual*, STMicroelectronics. [Online]. Available: <https://www.st.com/>
- [10] *RM0440: STM32G4 Series advanced Arm-based 32-bit MCUs – Reference manual*, STMicroelectronics. [Online]. Available: <https://www.st.com/>
- [11] *UM2637: Discovery kits with increased-frequency 800 MHz STM32MP157 MPUs – User manual*, Rev. 4, STMicroelectronics. [Online]. Available: <https://www.st.com/>
- [12] *SN65HVD23x 3.3-V CAN Bus Transceivers – Datasheet*, Literature number SLOS346, Texas Instruments. [Online]. Available: <https://www.ti.com/>
- [13] FreeRTOS. “FreeRTOS Real Time Kernel,” Accessed: Sep. 26, 2026. [Online]. Available: <https://www.freertos.org/>

Appendix A

Frame Layouts

All multi-byte values are big-endian. Bytes 6 and 7 of the cyclic frames are written by `E2e_Protect()` and checked by `E2e_Check()`.

Table A.1: Frame 0x100 SENSOR_SPEED (DLC 8, 10 ms)

Byte	Signal	Encoding
0–1	VehicleSpeed	unsigned, 0.1 km/h per bit, valid 0–2500
2–3	AdcRaw	12-bit raw ADC value (diagnostic)
4	Status	bit 0: fault injection active; bit 1: ADC read error
5	–	reserved, 0
6	AliveCounter	0–15 in the low nibble, +1 per frame
7	CRC	CRC-8 SAE-J1850 (poly 0x1D, init 0xFF, xor-out 0xFF) over bytes 0–6

Table A.2: Frame 0x101 SENSOR_ALIVE (DLC 8, 100 ms)

Byte	Signal	Encoding
0	NodeState	0 = normal, 1 = fault injection
1–4	Uptime	seconds since reset
5	BusOffCount	saturates at 255
6	AliveCounter	as above
7	CRC	as above

Example

A speed of 123.4 km/h with ADC value 2021, no status flags and alive counter 5 gives the bytes 04 D2 07 E5 00 00 05 CRC, where $0x04D2 = 1234$ and $0x07E5 = 2021$.

Appendix B

Build, Deployment and Diagnosis Procedures

B.1 Host unit tests

```
cd tests
make                # builds and runs test_isotp and test_e2e
```

Listing B.1: Running the unit tests

On Windows the MSYS2 UCRT64 `gcc` and `make` must be on the `PATH`; the makefile detects MinGW and disables the undefined-behaviour sanitiser, which MinGW does not provide.

B.2 Sensor ECU

The project in `sensor_ecu/` is built in STM32CubeIDE and flashed through the on-board ST-LINK, for example:

```
STM32_Programmer_CLI -c port=SWD mode=UR -w Debug/sensor_ecu.elf -v -rst
```

Listing B.2: Flashing the NUCLEO-G431RB

The log is printed on the ST-LINK virtual COM port at 115200 baud, 8N1.

B.3 Control ECU (Cortex-M4 on the DK1)

The DK1 boots Linux from the SD card with both BOOT switches ON. Connected to the PC through the USB-C OTG port, it exposes a network interface and is reachable at 192.168.7.1.

```
scp control_ecu_CM4.elf root@192.168.7.1:/lib/firmware/control_ecu.elf
ssh root@192.168.7.1
R=/sys/class/remoteproc/remoteproc0
echo control_ecu.elf > $R/firmware
echo start > $R/state          # "stop" to halt the M4
```

Listing B.3: Loading and starting the M4 firmware

The M4 log is printed on UART7 (Arduino connector D1 = TX, D0 = RX) at 115200 baud.

B.4 Reading M4 memory and registers from Linux

The technique used in Section 4.5: find the address of a variable in the ELF file, translate the M4 SRAM address (0x1002xxxx) to the physical alias seen by the A7 (0x3002xxxx) and read it through `/dev/mem`.

```
arm-none-eabi-nm control_ecu_CM4.elf | grep -E " (xTickCount|hfdcan1)$"
arm-none-eabi-gdb -batch -ex "print/x (int)&hfdcan1.State - (int)&hfdcan1"
control_ecu_CM4.elf
```

Listing B.4: Locating symbols and structure offsets

```
import mmap, os, struct
fd = os.open("/dev/mem", os.O_RDONLY | os.O_SYNC)
ram = mmap.mmap(fd, 0x10000, mmap.MAP_SHARED, mmap.PROT_READ, offset=0
    x30020000)
can = mmap.mmap(fd, 0x1000, mmap.MAP_SHARED, mmap.PROT_READ, offset=0
    x4400E000)
rd = lambda m, off: struct.unpack("<I", m[off:off + 4])[0]
print("xTickCount =", rd(ram, 0x10025104 - 0x10020000))
print("FDCAN CCCR = 0x%08x" % rd(can, 0x18))
```

Listing B.5: Reading a 32-bit value from M4 SRAM or a peripheral (run on the DK1)

Peripheral registers of a stopped M4 may be unlocked; reading them then raises a bus error. Oscillator-disable registers of the RCC are secure-only when `RCC_TZCR.TZEN = 1` and cannot be written from Linux.